

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R

Testes de API

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Integração com CI/CD, padrões e boas práticas

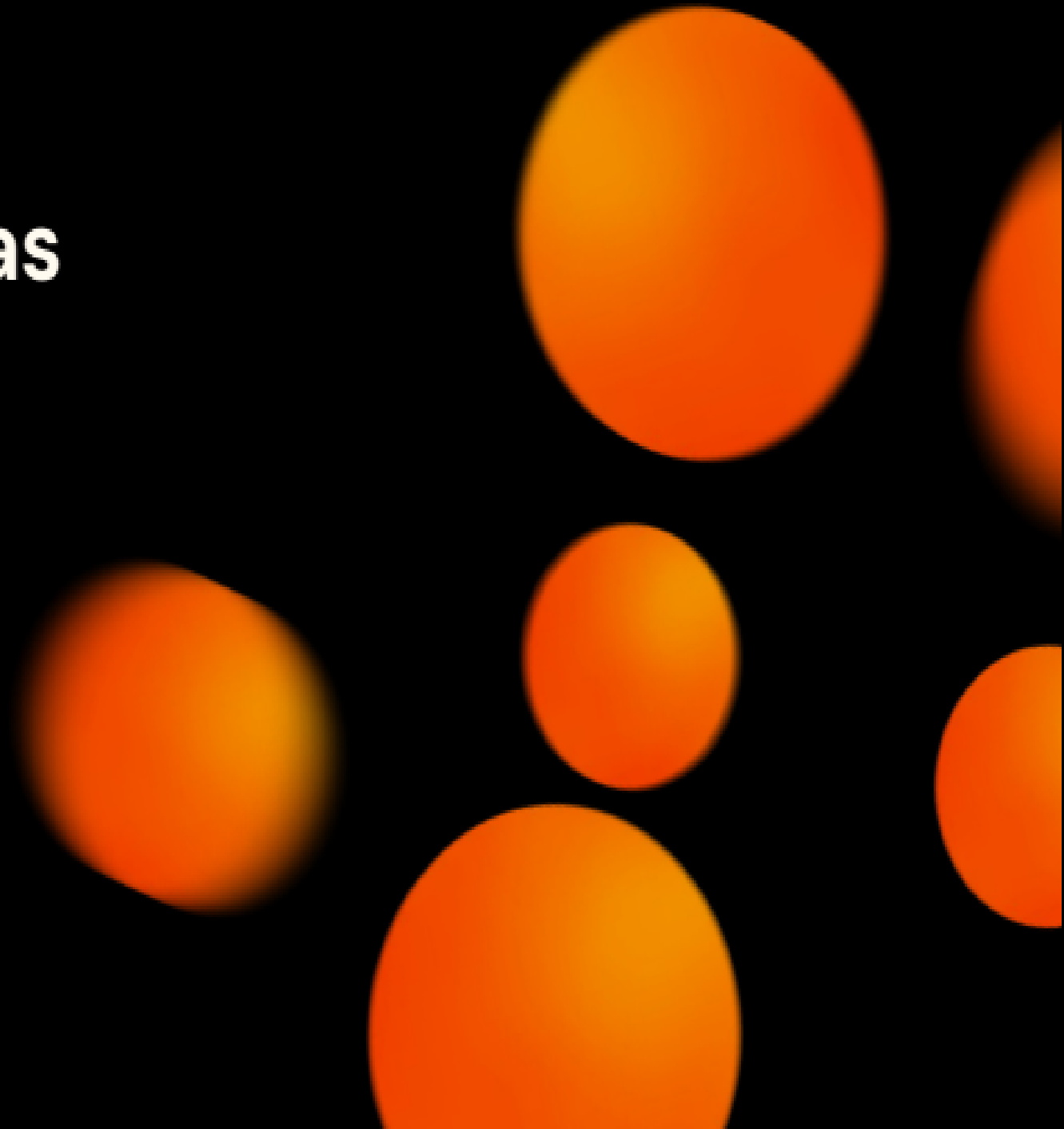
Testes de API
Aula 3



O que veremos hoje?

- Revisão das aulas anteriores
- Padrões e boas práticas na Automação
- CI e sua integração com a qualidade
- Como executar os testes automatizados fora da IDE?
- Testes de API e ferramentas de CI/CD
- Próximos passos no aprendizado

Revisão das aulas anteriores

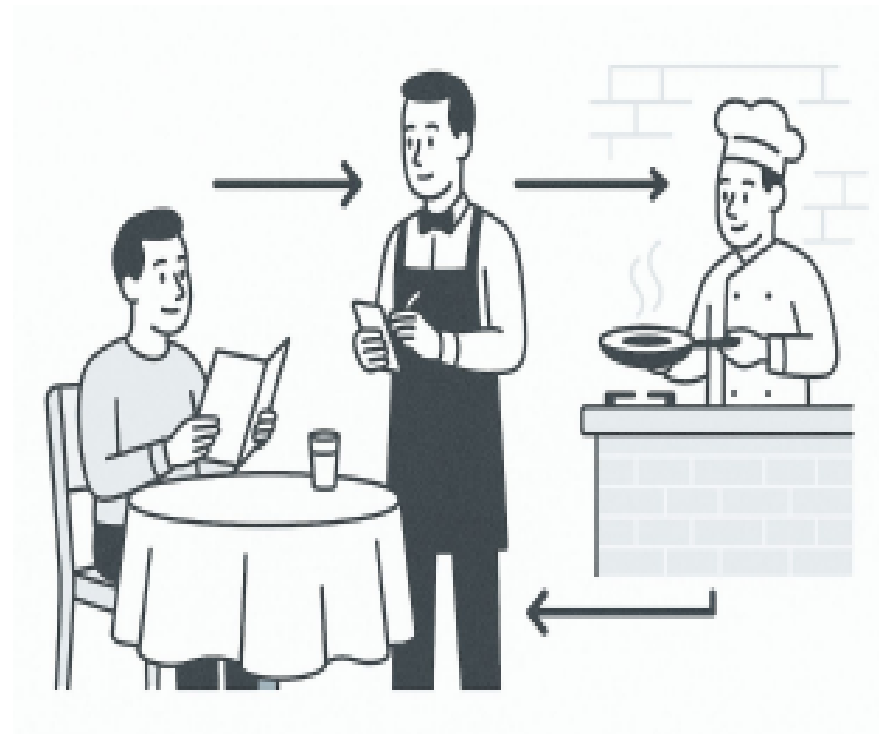


Revisão

→ API é um **conjunto de regras e protocolos** que permite que diferentes softwares se comuniquem sem precisar saber como eles foram implementados.

É um **contrato** que define claramente:

- O que podemos pedir
- Como devemos pedir
- O que vamos receber



Revisão

O que é o Postman?

O Postman é uma ferramenta completa bastante popular e intuitiva para trabalhar com APIs.



POSTMAN

Revisão

Para que serve?

- **Exploração Rápida:** Enviar requisições (pedidos) para APIs e ver as respostas instantaneamente.
- **Testes Manuais:** Verificar funcionalidades, erros, e dados de forma ágil.
- **Documentação:** Organizar as requisições em "Coleções", facilitando o reuso e o compartilhamento.
- **Automação:** Escrever scripts básicos para validar respostas automaticamente e encadear requisições.

Por que é tão popular?

- **Interface Amigável:** Fácil de usar, mesmo para quem está começando.
- **Versatilidade:** Suporta diversos tipos de requisições (GET, POST, PUT, DELETE) e autenticações.
- **Colaboração:** Facilita o trabalho em equipe, compartilhando coleções e ambientes.

Revisão

Quadro resumo das requisições HTTP, seus códigos e validações recomendadas:

Tipo	Status Codes	O que validar	Boas práticas
GET	200, 404	Dados, estrutura, tempo, headers	Validar positivos e negativos
POST	201, 400	Criação, ID gerado, persistência	Campos obrigatórios, dados inválidos
PUT	200, 204, 400, 404	Atualização correta, idempotência	Atualizar existentes e inválidos
DELETE	204, 200, 404	Remoção, idempotência, persistência após DELETE	Deletar existentes e já deletados

Revisão

RestAssured é uma biblioteca Java para testes de APIs REST. Ela facilita a criação de testes automatizados HTTP com suporte a:

- Métodos **GET**, **POST**, **PUT**, **DELETE**, etc.
- Validação de status, cabeçalhos e body
- Integração com **JUnit** ou **TestNG**
- Autenticação (Basic, OAuth, etc)

Porque usar o RestAssured?

- Sintaxe fluida e legível (estilo Gherkin)
- Ideal para QA/DEVs que usam Java e precisam testar APIs
- Boa integração com frameworks de testes e CI/CD

Revisão

Dicas Extras Gerais

- Testar **autenticação** e **autorização**: requisições sem token ou com token inválido: 401/403 
- Testar **timeout** e **desempenho** (respostas rápidas e estáveis) 
- Testar **limites** da API: tamanho máximo de payload, quantidade de requests (rate limit) 
- Testar **headers** obrigatórios: Content-Type, Accept, Authorization 
- Validar **consistência** entre operações (Ex.: POST → GET → DELETE → GET) 

Autenticação em testes automatizados

Autenticação em API

Autenticação em APIs é o processo de garantir que quem faz uma requisição é uma entidade autorizada

→ APIs frequentemente requerem autenticação para proteger dados sensíveis e restringir acessos não autorizados.

Principais Tipos de Autenticação em APIs

1 Sem Autenticação (Pública)

- API aberta, qualquer um pode consumir.
 - Ex.: APIs de dados públicos como OpenWeatherMap, IBGE.

Autenticação em API

2 Basic Authentication (Autenticação Básica)

- Usa uma combinação de **usuário + senha**, codificados em **Base64**.
- Enviados no cabeçalho HTTP:

`Authorization: Basic <Base64(usuario:senha)>`

Exemplo RestAssured - Basic Auth

- `given().`
- `auth().preemptive().basic("usuario, "senha").`
- `when().`
- `get("/minha-api").`
- `then().`
- `statusCode(200);`

 **Problema:** pouco segura sem HTTPS.

Autenticação em API

3 API Key (Chave de API)

- A chave é enviada na URL ou no header.

`GET /produtos?apikey=123456789`

ou

`Authorization: ApiKey 123456789`

Exemplo RestAssured - API Key

- `given().`
- `queryParam("apikey", "123456789").`
- `when().`
- `get("/minha-api").`
- `then().`
- `statusCode(200);`

✓ Simples de implementar.

✗ Menos segura se não for usada sobre HTTPS.

Autenticação em API

4 Token (JWT, Bearer Token)

- Cliente faz login (POST com usuário/senha).
- API responde com um **token** (geralmente JWT).
- As próximas requisições incluem esse token no header:

- Authorization: Bearer <colar_o_token_aqui>

Exemplo RestAssured - Basic Auth

- given().
- header("Authorization", "Bearer colar_o_token_aqui").
- when().
- get("/minha-api").
- then().
- statusCode(200);

✓ Mais seguro, mais comum em APIs modernas.

Autenticação em API

Quadro comparativo dos tipos de Autenticação:

Tipo	Segurança	Facilidade	Uso Comum
Pública	✗	✓	Dados públicos
Basic Auth	⚠	✓	APIs internas, legadas
API Key	⚠	✓	APIs simples, SaaS
Token (JWT)	✓	⚠	APIs modernas, microserviços

Autenticação em API

Exemplo de Método Utilitário para Token:

```
○ public String getAuthToken() {  
○     return  
○         given().  
○             contentType("application/json").  
○             body("{ \"email\": \"user@test.com\", \"password\": \"1234\" }").  
○             when().  
○                 post("/login").  
○             then().  
○                 statusCode(200).  
○                 extract().  
○                 path("authorization");  
○ }
```

Autenticação em API

Uso do Método Utilitário no teste:

- `String token = getAuthToken();`
-
- `given().`
- `header("Authorization", token).`
- `when().`
- `get("/produtos").`
- `then().`
- `statusCode(200);`

Autenticação em API

Boas práticas com autenticação em testes:

-  Nunca expor senhas ou tokens fixos no código.
-  Gerar tokens de forma dinâmica antes dos testes.
-  Validar que endpoints sem autenticação retornam corretamente **401 Unauthorized** ou **403 Forbidden**.

Organização e boas práticas na Automação

Para que seus testes automatizados sejam realmente um ativo e não um fardo, a **organização** e a adoção de **boas práticas** são fundamentais. O objetivo é manter os testes:

- Fáceis de entender
- Fáceis de manter
- Fáceis de escalar

Nomeação e Organização Clara

→ **Nomeação Clara:** Dê nomes descritivos e consistentes a testes, funções e arquivos (ex: `test_cadastro_usuario_sucesso`, `validarStatus201`).

→ **Organização em Suites:** Agrupe testes relacionados em "suites" (ex: `health-check`, `contrato`, `funcional`, `segurança`) para facilitar a execução e o entendimento.

Gerenciamento de Ambientes e Dados

- **Separação de Ambientes:** Utilize configurações distintas para diferentes ambientes (dev, QA, staging, prod), usando variáveis e arquivos específicos.
- **Uso de Arquivos de Configuração:** Armazene URLs, tokens e dados sensíveis em arquivos de configuração separados do código dos testes, evitando *hard code*.
- **Dados Dinâmicos e Reutilização:** Evite dados fixos ("hard code"). Crie dados dinamicamente ou use dados reutilizáveis para garantir a independência de cada teste.

Design de Testes Robustos

- **Testes Independentes:** Cada teste deve ser capaz de rodar sozinho, sem depender do sucesso ou falha de outros testes.
- **Testes Consistentes:** Rodar o mesmo teste várias vezes deve resultar sempre no mesmo estado final do sistema (se aplicável, para PUT/DELETE).
- **Testes Resilientes:** Testes devem ser menos sensíveis a pequenas mudanças na UI ou no ambiente, focando na lógica da API.

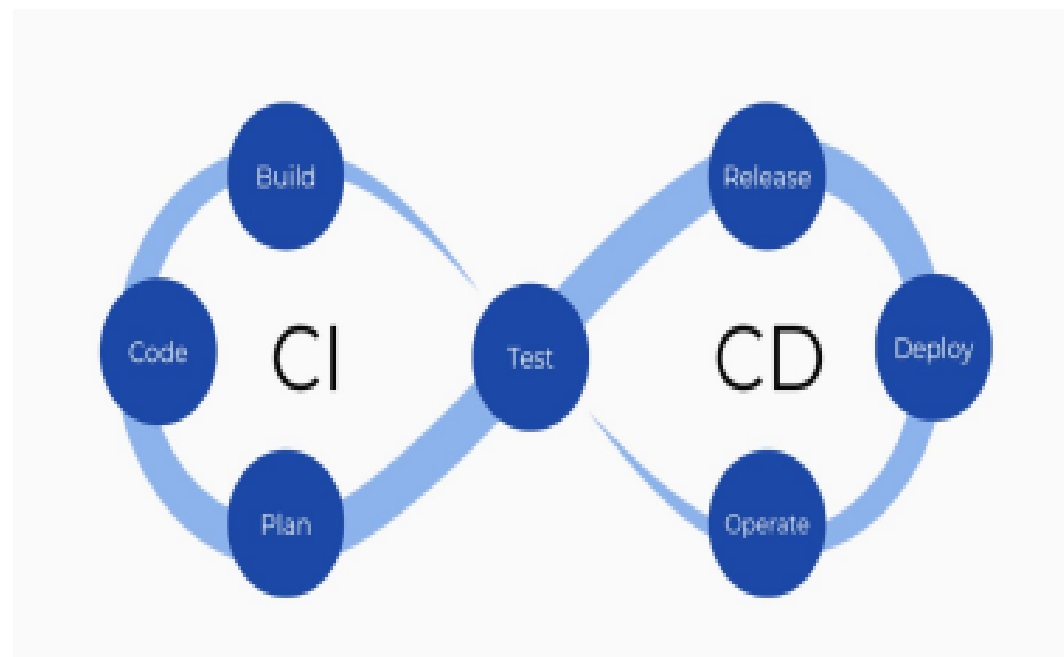
Monitoramento e Feedback

- **Logging:** Implemente logs detalhados para registrar o que acontece durante a execução dos testes, auxiliando na depuração de falhas.
- **Relatórios:** Gere relatórios claros e concisos que mostram o status da execução (passou/falhou), facilitando a análise e a tomada de decisão.

Introdução à integração contínua e importância nos testes de API

O Que é Integração Contínua (CI)?

- É a prática de desenvolvedores **integrarem seu código** em um **repositório central** **várias vezes ao dia**.
- Cada integração desencadeia um **build automatizado** e a **execução de testes automatizados** para verificar o código.
- **Objetivo:** Detectar e resolver problemas de integração e *bugs* o mais cedo possível.



A Importância da CI nos Testes de API

- **Feedback Rápido:** Testes de API são rápidos e estáveis, perfeitos para serem executados em cada "push" de código. Isso nos dá um feedback quase instantâneo sobre a saúde da API.
- **"Cinto de Segurança":** A cada alteração, a CI garante que as funcionalidades existentes da API não foram quebradas.
- **Confiança:** A execução constante de testes de API no pipeline de CI aumenta a confiança de toda a equipe no funcionamento do serviço.

Como a CI Melhora a Qualidade em QA?

- **Deteccção Precoce de Bugs:** Problemas são encontrados no momento em que são introduzidos, reduzindo o custo e a complexidade da correção.
- **Menos Regressões:** A automação constante minimiza a chance de novas funcionalidades quebrarem o que já funcionava.
- **Foco Estratégico para QA:** Com os testes automatizados básicos rodando na CI, o time de QA pode dedicar mais tempo a testes exploratórios, cenários complexos, usabilidade e segurança aprofundada.
- **Colaboração Aprimorada:** A CI força a equipe a integrar e testar frequentemente, promovendo uma cultura de qualidade compartilhada entre devs e QA.

Ferramentas Populares



Azure Pipelines

Não há "melhor" ferramenta absoluta!

A melhor ferramenta é aquela que se adapta melhor ao contexto da equipe e do projeto.

Executar uma coleção de testes via linha de comando

Executando Testes do Postman com Newman

→ O Newman é uma ferramenta de linha de comando que permite rodar suas coleções do Postman. Para usá-lo, precisamos ter a coleção e ambiente exportados.

O que vamos precisar:

- Sua **Coleção Postman** (com os testes JavaScript na aba "Tests").
- O **Newman** instalado na sua máquina ou servidor.
- Um **Terminal** ou Prompt de Comando.

Exporte Sua Coleção e Ambiente do Postman Web

→ Exportando a Coleção:

- Acesse o Postman Web (go.postman.co/home).
- Selecione o seu Workspace (onde está localizada a sua Collection)
- No menu lateral esquerdo, vá para a aba **"Collections"**.
- Passe o mouse sobre a coleção que deseja exportar e clique no ícone de **três pontinhos (...)** ou no ícone de **"..."** ao lado do nome.
- Selecione **"Export"**.
- Escolha a versão de exportação (geralmente a mais recente, ex: "Collection v2.1").
- Clique em **"Export"** novamente e salve o arquivo **.json** em um local fácil de encontrar na sua máquina (ex: **minha_colecao.json**).

Exporte Sua Coleção e Ambiente do Postman Web

→ Exportando o Ambiente:

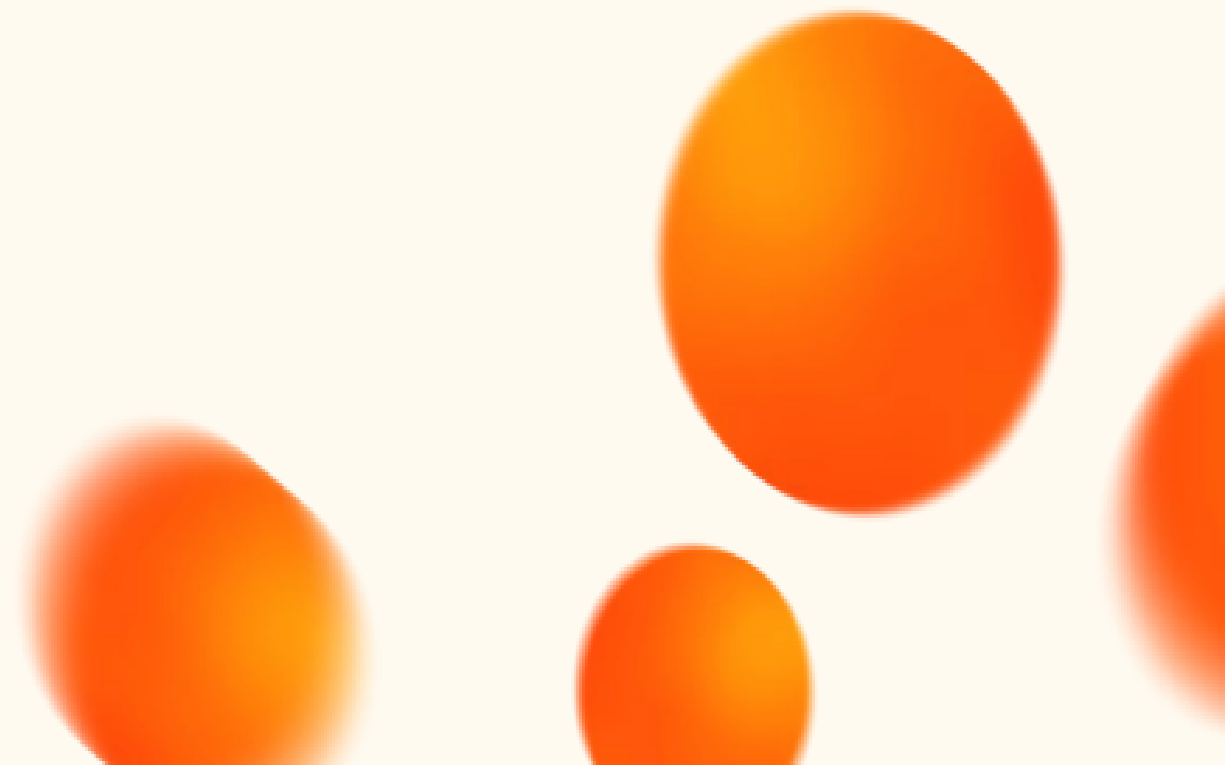
- Como estamos usando variáveis de ambiente (como `baseUrl` para a `ServerRest`), vamos precisar exportar o ambiente.
- No Postman Web, vá para a aba "**Environments**" no menu lateral esquerdo (ou clique no ícone de engrenagem no canto superior direito e selecione "Manage Environments").
- Passe o mouse sobre o ambiente que deseja exportar e clique no ícone de **três pontinhos (...)**.
- Selecione "**Export**". Salve o arquivo `.json` (ex: `meu_ambiente_dev.json`).

INTERVALO!



20 min.

Você instalou o Node.js?



Instalando o Node JS

→ Tutorial de instalação para Windows

- # Se tiver o Chocolatey:
- choco install nodejs-lts

→ Tutorial de instalação para Linux

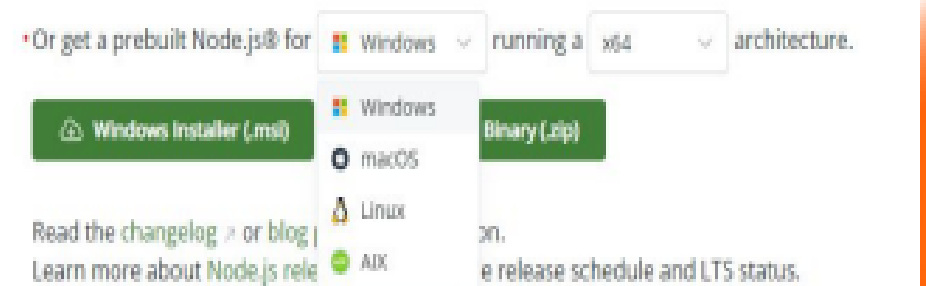
- # Atualiza pacotes
- sudo apt update
- # Instala Node.js e npm direto do repositório oficial
- sudo apt install -y nodejs npm

→ Tutorial de instalação para MacOS

- # Instale o homebrew com:
- /bin/bash -c "\$(curl -fsSL <https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh>)"
- # Instalar Node.js pelo Homebrew
- brew install node

Você também pode baixá-lo de

<https://nodejs.org/pt-br/download/> basta selecionar seu sistema operacional e clicar na opção de Installer:



Instalando o Newman

→ O Newman é uma ferramenta baseada em Node.js. Como acabamos de instalar o Node, abra seu Terminal (no Windows: Prompt de Comando ou PowerShell; no macOS/Linux: Terminal) e execute o seguinte comando:

```
npm install -g newman
```

Ou **MELHOR**, instale o Newman e o HTML Reporter (veremos sua função em breve) **juntos**, com o comando:

```
npm install -g newman newman-reporter-htmlextra
```

- Estes comandos instalam o Newman globalmente na sua máquina, o que permite que você o execute de qualquer diretório.

Executando Sua Coleção com Newman

→ **Navegue até o diretório:** No seu Terminal, navegue até a pasta onde você salvou o arquivo `.json` da sua coleção (e do ambiente). Use o comando `cd` para isso.

- Exemplo: `cd C:\Users\SeuUsuario\Documentos\TestesAPI` ou `cd ~/Documentos/TestesAPI`

→ **Comando Básico para Executar a Coleção:**

```
newman run minha_colecao.json
```


Executando Sua Coleção com Newman

→ Comando para Executar a Coleção com um Ambiente:

Se você exportou um arquivo de ambiente e quer usá-lo, inclua a flag `-e` (ou `--environment`) seguida do nome do arquivo do ambiente:

```
newman run minha_colecao.json -e meu_ambiente_dev.json
```

O Newman executará todas as requisições na sua coleção, incluindo os scripts de teste que você configurou na aba "Tests", e mostrará um relatório detalhado diretamente no terminal.

Resultado da Execução

ServeRest		
+ GET usuários		
GET https://serverest.dev/usuarios [200 OK, 34.39kB, 646ms]		
✓ Status da requisição é 200 OK		
✓ Resposta é um array (lista)		
+ POST usuários		
POST https://serverest.dev/usuarios [201 Created, 548B, 289ms]		
✓ Status da requisição é 201 Created		
✓ Mensagem de sucesso ao cadastrar usuário		
✓ ID do usuário capturado		
+ PUT usuários		
PUT https://serverest.dev/usuarios/PeXA155nNXBpVHi3 [200 OK, 583B, 214ms]		
✓ Status da requisição é 200 OK		
✓ Mensagem de sucesso ao atualizar usuário		
+ DELETE usuários		
DELETE https://serverest.dev/usuarios/PeXA155nNXBpVHi3 [200 OK, 584B, 214ms]		
✓ Status da requisição é 200 OK		
✓ Mensagem de sucesso ao remover usuário		
	executed	failed
Iterations	1	0
requests	4	0
test-scripts	4	0
prerequisite-scripts	0	0
assertions	9	0
total run duration: 1354ms		
total data received: 34.11kB (approx)		
average response time: 328ms (min: 289ms, max: 646ms, s.d.: 187ms)		

Geração de Relatórios HTML

→ Para ter relatórios mais visuais, que podem ser abertos no navegador, é possível instalar um "reporter" adicional como o **htmlextra**:

Para instalar o HTML Reporter:

```
npm install -g newman-reporter-htmlextra
```

Alternativamente, como já vimos antes, é possível instalar o Newman e o HTML Reporter **juntos**, com o comando:

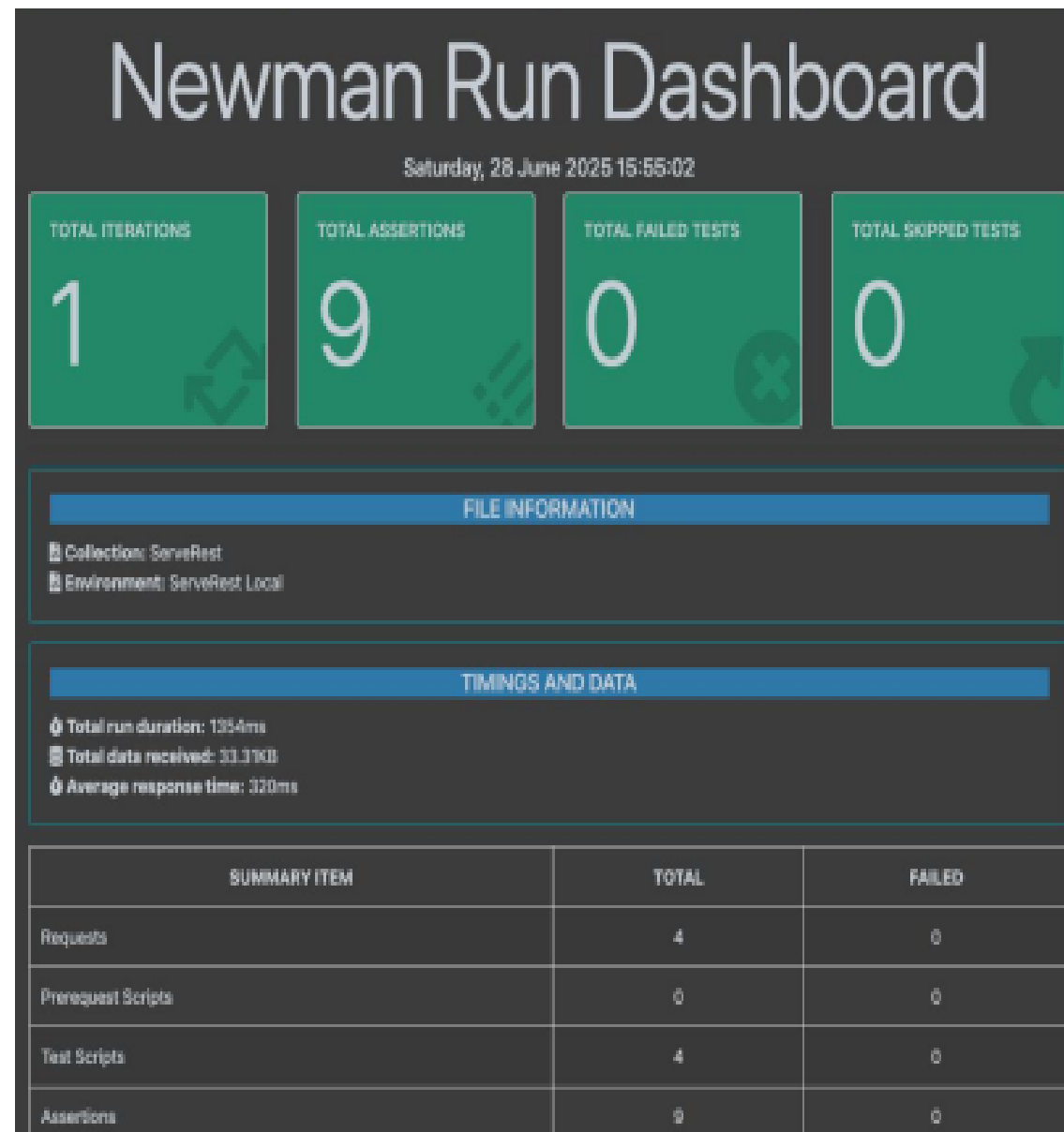
```
npm install -g newman newman-reporter-htmlextra
```

Execute com Relatório HTML:

```
newman run minha_colecao.json -e meu_ambiente_dev.json --reporters "cli,htmlextra"
```

Este comando executará os testes e gerará um arquivo HTML (geralmente **newman-report.html**) na pasta onde você executou o comando. Você pode abrir esse arquivo no seu navegador para ver os resultados de forma mais gráfica.

Relatório HTML



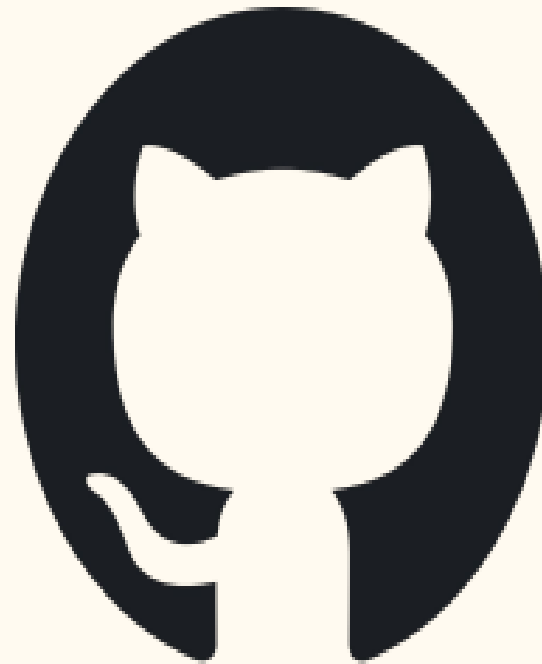


Hora do Quiz! 🧐



Integrando testes de API com pipelines

Já possui conta no GitHub Actions?



Link do [repositório](#) construído em aula

Integrando Testes de API ao GitHub Actions

- O GitHub Actions funciona com arquivos de configuração (.yml ou .yaml) que você coloca no seu próprio repositório.
- Suba esses arquivos para o seu repositório GitHub. Uma boa prática é criar uma pasta específica para eles, por exemplo: seu-projeto/tests/postman/.

```
seu-projeto/  
├── .github/  
│   └── workflows/  
│       └── testes-api.yml  <-- Vamos criar este arquivo!  
├── tests/  
│   └── postman/  
│       ├── minha_colecao.json  
│       └── meu_ambiente_dev.json  
└── (outros_arquivos_do_seu_projeto)
```


Integrando Testes de API ao GitHub Actions

→ Agora, vamos criar o arquivo que o GitHub Actions vai ler para saber o que fazer.

- No seu repositório GitHub (via interface web ou localmente), crie as pastas: `.github/workflows/`.
- Dentro da pasta `workflows`, crie um novo arquivo chamado, por exemplo, `testes-api.yml`.
- Cole o seguinte conteúdo YAML dentro desse arquivo:

[Código YML aqui](#)

Observação: Importante lembrar de manter os nomes dos arquivos `.json` da Collection e Environment de acordo com os nomes especificados no `#passo5` do código do `.yml`.

Integrando Testes de API ao GitHub Actions

→ Explicação do Código YAML:

- **name:** É o nome do seu pipeline.
- **on:** Define quando o pipeline será acionado. Aqui, ele rodará em cada **push** de código e em cada **pull_request** (abertura/atualização).
- **jobs:** Um pipeline pode ter um ou mais "jobs". O nosso é **rodar-testes-newman**.
 - **runs-on: 'ubuntu-latest':** Indica que o job será executado em uma máquina virtual Linux fornecida pelo GitHub.
 - **steps:** São as ações sequenciais dentro do job:
 - **actions/checkout@v4:** Uma "action" pré-definida do GitHub que baixa seu código para a máquina virtual.
 - **actions/setup-node@v4:** Instala o Node.js, que é necessário para o Newman.
 - **npm install -g newman:** Instala o Newman globalmente na máquina virtual.
 - **npm install -g newman-reporter-htmlextra (Opcional):** Instala o reporter para gerar o HTML visual.
 - **run: | newman run ...:** Este é o comando chave! Ele executa o Newman com sua coleção e ambiente. **Lembre-se de ajustar os caminhos dos arquivos .json** para onde eles estão no seu repositório. O **|** no final da primeira linha indica que o comando continua na próxima linha no YAML.
 - **actions/upload-artifact@v4:** Uma "action" que permite você "publicar" arquivos gerados pelo pipeline (como o relatório HTML) para que possam ser baixados da interface do GitHub.

Integrando Testes de API ao GitHub Actions

→ **Faça o Push:** Após salvar o arquivo `testes-api.yml` no diretório `.github/workflows/` e ter seus arquivos `.json` da coleção e ambiente no local correto (ex: `tests/postman/`), faça um `git push` para o seu repositório GitHub.

→ **Monitore na Aba "Actions":**

- Vá para o seu repositório no GitHub no navegador.
- Clique na aba **"Actions"** (Ações).
- Você verá um workflow com o nome "Executar Testes de API com Newman" (ou o nome que você deu) sendo executado. Clique nele para ver os detalhes.
- Dentro da execução, você pode clicar nos "jobs" e "steps" para ver os logs em tempo real.
- Se o relatório HTML foi gerado e publicado (Passo 6 no YAML), você o encontrará na página de resumo da execução do workflow, na seção "Artifacts" (Artefatos), para download.

Integrando Testes de API ao GitHub Actions

Summary
Jobs
rodar-testes-newman
Run details
Usage
Workflow file

rodar-testes-newman

completed 15 minutes ago in 12s

Executar Testes de API

```

1  # Run newman run ./test/pastel/servevent.persona_collection.json --e ./test/pastel/servevent_local.pastel_environment.json \
2  newman
3
4  # Servidor
5
6  # GET usuarios
7  GET https://servevent.developersapi.com/usuarios [200 OK, 34.84KB, 21ms]
8  - Status de resposta é 200 OK
9  - Resposta é um array (lista)
10
11 # POST usuarios
12 POST https://servevent.developersapi.com/usuarios [201 Created, 54KB, 11ms]
13 - Status de resposta é 201 Created
14 - Mensagem de sucesso ao cadastrar usuário
15 - ID de usuário capturado
16
17 # PUT usuarios
18 PUT https://servevent.developersapi.com/usuarios/1 [200 OK, 30ms, 7ms]
19 - Status de resposta é 200 OK
20 - Mensagem de sucesso ao atualizar usuário
21
22 # DELETE usuarios
23 DELETE https://servevent.developersapi.com/usuarios/1 [200 OK, 54KB, 7ms]
24 - Status de resposta é 200 OK
25 - Mensagem de sucesso ao remover usuário
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53

```

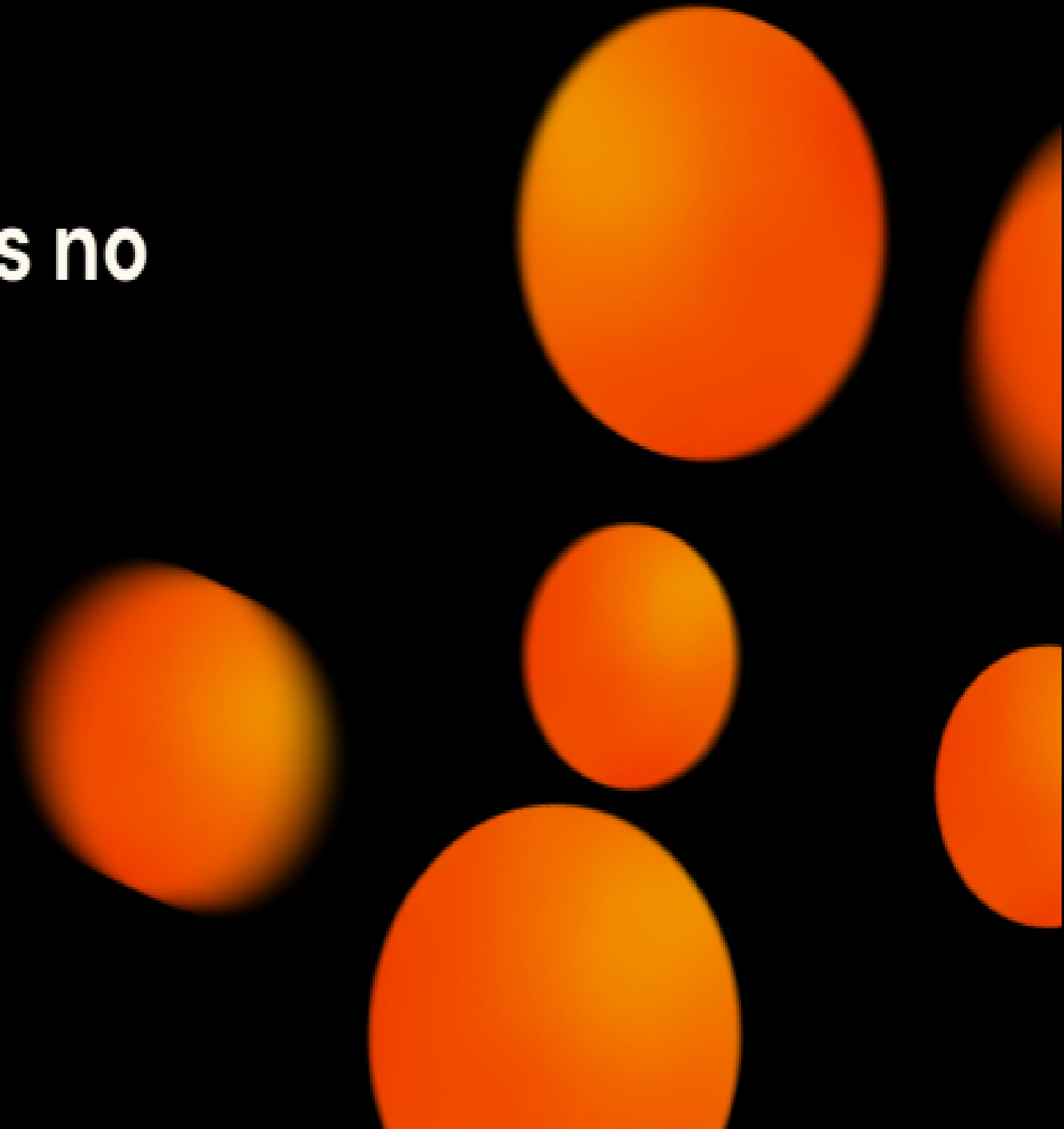
	passed	failed
iterations	1	0
requests	4	0
test-scripts	4	0
prerequest-scripts	0	0
assertions	5	0

total run duration: 60ms

total data received: 34.87KB (logged)

average response times: 11ms (min: 7ms, max: 20ms, s.d.: 7ms)

Próximos passos no aprendizado



Desafios Práticos na Automação de APIs

- **Mudanças Constantes na API:** APIs evoluem rapidamente, e adaptar os testes a essas alterações pode ser um desafio contínuo.
- **Manutenção de Testes "Quebrados":** Testes que falham intermitentemente (flaky tests) ou que se tornam obsoletos exigem atenção constante e manutenção.
- **Dados Sensíveis:** Lidar com a segurança e a gestão de dados confidenciais (tokens, senhas, informações de usuários) durante a automação.
- **Ambiente Instável:** Flutuações nos ambientes de teste (dev, QA) podem causar falhas nos testes, mesmo que a API esteja funcional.
- **Integrações Complexas:** APIs que dependem de muitos outros serviços (microserviços) aumentam a complexidade dos cenários de teste.

Próximos Passos no Seu Aprendizado

→ Aprofundar em Automação

Explorar frameworks mais avançados (ex: Cypress para API, Playwright, Karate DSL).

→ Testes de Performance

Ferramentas dedicadas (JMeter, K6) para análises mais profundas de carga e estresse.

→ Testes de Segurança Aprofundados

Conhecer ferramentas de SAST/DAST e conceitos de OWASP Top 10 para APIs.

→ Testes de Contrato (Avançado)

Ferramentas como Pact para garantir a compatibilidade entre serviços.

→ Monitoramento e Observabilidade

Aprender a monitorar APIs em produção para identificar problemas proativamente.

→ Engenharia de Qualidade (QE)

Evoluir para uma mentalidade de "qualidade embutida" em todo o ciclo de desenvolvimento.

Recomendações de estudo

- **Ferramentas:** Newman, Postman CLI, RestAssured avançado, Karate, Pact (Contract Testing).
- **Livros:** "Continuous Testing for DevOps Professionals", "Testing Web APIs".
- **Certificações:** Postman, API Testing - Udemy, ISTQB Test Automation.

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados